Empirical Software Engineering (2026) 31:3
https://doi.org/10.1007/s10664-025-10749-4

A zero-shot framework for cross-project vulnerability detection in source code

Radowanul Haque[1] · **Aftab Ali**[1] · **Sally McClean**[1] · **Naveed Khan**[1]

Received: 18 February 2025 / Accepted: 7 October 2025

Abstract
The growing prevalence of software vulnerabilities has increased the need for effective detection methods, particularly in cross-project settings where domain differences create significant challenges. Existing vulnerability detection models often struggle to generalise across projects due to variations in coding styles, feature distributions, and the absence of labelled target data. This paper presents ZSVulD, a zero-shot, cross-project vulnerability detection framework designed to operate without target-domain labels. ZSVulD uses domain-agnostic CodeBERT embeddings to capture both syntactic and semantic features of source code, enabling knowledge transfer between projects. The framework applies an iterative pseudo-labelling process in which a neural network and XGBoost classifier collaboratively refine predictions for the target domain. Feature alignment is incorporated as a diagnostic technique to assess and visualise distributional differences between source and target datasets. Experiments on the Devign and REVEAL datasets show that ZSVulD achieves higher recall, F1, and F2 scores compared to existing methods, with an emphasis on reducing false negatives. These findings indicate that ZSVulD can support automated vulnerability detection pipelines, contributing to more reliable security assessments across different software projects.

Keywords LLMs · Vulnerability detection · Code classification · Zero-Shot learning

Communicated by Chunyang Chen, James Zheng

✉ Radowanul Haque
haque-r1@ulster.ac.uk

Aftab Ali
a.ali@ulster.ac.uk

Sally McClean
si.mcclean@ulster.ac.uk

Naveed Khan
n.khan@ulster.ac.uk

[1] School of Computing, Ulster University, Belfast BT15 1ED, U.K.

Published online: 29 October 2025

1 Introduction

The increasing reliance on software in critical systems has brought heightened attention to the discovery and mitigation of software vulnerabilities. Vulnerability detection, the process of identifying weaknesses or flaws in software that could be exploited to compromise security, integrity, or functionality, has become a cornerstone of research in software security. These vulnerabilities pose significant risks, ranging from data breaches to system failures, making their detection critical to protecting sensitive data and ensuring system reliability (Chakraborty et al. 2022). According to a 2024 study by IBM, the global average cost of a data breach reached $4.88 million, with breaches in critical infrastructure sectors incurring even higher costs (IBM 2024).

Recent surveys (Kaur and Nayyar 2020) (Zeng et al. 2020; Chakraborty et al. 2022; Harzevili et al. 2023) on software vulnerability detection provides a comprehensive overview of traditional and modern techniques, from static and dynamic analysis to machine learning-based approaches. Static analysis tools like Flawfinder (David and Wheeler 2017) rely on rule-based heuristics, while dynamic analysis tools such as fuzzing identify vulnerabilities by monitoring software execution (Kaur and Nayyar 2020). However, these methods often face challenges related to scalability and the generation of false positives, necessitating the exploration of advanced, learning-based techniques. The survey by Lin et al. (Lin et al. 2020) delves into deep neural network applications for vulnerability detection, highlighting their potential for learning complex patterns in source code. Similarly, a systematic review by Harzevili et al. (Harzevili et al. 2023) discusses machine learning advancements in automated vulnerability detection, emphasising the importance of dataset quality, feature engineering, and explainability. Marjanov et al. (Marjanov et al. 2022) analyse what works well in machine learning for source code vulnerability detection and identify gaps in current approaches, such as addressing complex code semantics and dataset biases.

Within-project vulnerability detection has shown promising results by training and evaluating models on data from the same software project. However, these models typically struggle when applied to previously unseen projects. This shortcoming stems from substantial domain shifts between projects, including variations in coding conventions, library dependencies, architectural patterns, and project-specific constraints. Such differences lead to distribution mismatches between the source and target data, which hinder the generalisability of learned representations. In practice, this presents a significant barrier to the adoption of learning-based vulnerability detection systems, as real-world deployments often involve analysing unfamiliar or proprietary codebases without any labelled data. Annotating new datasets for each project is costly and time-consuming, particularly given the expertise required to label security-critical code. Recent studies have therefore highlighted the importance of cross-project or zero-shot detection methods, which aim to transfer knowledge from well-annotated source projects to new targets without relying on target-side supervision (Li et al. 2023a).

Zero-shot and few-shot learning in Cross-Project vulnerability Detection (CPVD) (Nguyen et al. 2020, 2024; Li et al. 2023a) have gained significant attention as promising strategies to address these challenges. CPVD aims to train models on one or more source projects and apply them to detect vulnerabilities in the target without requiring labelled target data, making it particularly well-suited for zero-shot learning (Li et al. 2023a). However,

this task is inherently complex due to domain discrepancies between source and target datasets arising from differences in coding practices, libraries, and project-specific conventions.

Recent studies (Lin et al. 2018; Nguyen et al. 2020, 2024; Liu et al. 2022; Li et al. 2023a; Zhang et al. 2023; Cai et al. 2024) have explored various domain adaptation techniques to bridge these gaps, leveraging transfer learning, adversarial feature alignment, and graph-based neural networks to capture and transfer vulnerability-relevant patterns. Despite these advancements, existing methods often face three major limitations: (1) difficulty in aligning feature distributions between source and target projects, (2) noisy pseudo-labels that hinder adaptation performance, and (3) inefficient integration of syntactic and semantic representations, reducing their ability to generalise effectively in zero-shot settings.

To address these challenges, this paper introduces ZSVulD, a zero-shot framework for cross-project vulnerability detection. Code functions from both source and target datasets are embedded using a frozen CodeBERT model (Feng et al. 2020), and mean-pooled to obtain fixed-length vectors that capture syntactic and semantic features. These embeddings are normalised for consistency across domains. A neural network is trained on the source data and used to assign pseudo-labels to the unlabelled target data. The model is retrained over three iterations using a combination of source and pseudo-labelled target samples to enhance adaptation. After the final iteration, confidence scores are used to compute sample weights, which guide the training of three XGBoost classifiers. The final prediction is obtained by averaging their outputs, improving robustness under domain shift without requiring any labelled target data.

The main contributions of this work are as follows.

1. To the best of our knowledge, we provide the first systematic analysis of the limitations of CodeBERT (Feng et al. 2020) and UniXcoder (Guo et al. 2022) in zero-shot, cross-project vulnerability detection.
2. We introduce an iterative pseudo-labelling strategy with ensemble refinement that is robust to noisy pseudo-labels and mitigates distribution mismatch across domains in zero-shot settings.
3. We propose ZSVulD, a practical ensemble-based framework that uses CodeBERT embeddings, iterative pseudo-labelling, and ensemble refinement for zero-shot, cross-project vulnerability detection.

The remainder of this paper is structured as follows: Section 2 presents the literature review, Section 3 details the proposed framework, Section 4 describes the experimental settings, Section 5 presents the results, Section 6 describes the discussion and failure analysis, Section 7 describes threats to validity, and Sect. 8 concludes with future works.

2 Related works

The application of machine learning (ML) and deep learning (DL) techniques has transformed vulnerability detection in source code. These approaches have shifted the paradigm from static, rule-based methods to data-driven frameworks capable of uncovering complex patterns indicative of vulnerabilities. Two prominent paradigms have emerged in this context: within-project detection, which focuses on identifying vulnerabilities within a single

project, and cross-project detection, which seeks to generalise detection capabilities across diverse projects. This section reviews existing research, emphasising the contributions, methodologies, and challenges in leveraging ML and DL for these tasks.

2.1 Within-Project vulnerability detection

Vulnerability detection in source code is a critical aspect of software security, aiming to identify weaknesses that could be exploited by attackers. Recent advancements in machine learning (ML), deep learning (DL), natural language processing (NLP), and graph-based learning have significantly improved the precision and scalability of these techniques.

Traditionally, vulnerability detection relied heavily on rule-based static and dynamic analysis methods (Kaur and Nayyar 2020). While effective in specific contexts, these methods are often limited by their inability to generalize across diverse codebases and their susceptibility to false positives. Recent research addresses these limitations by incorporating data-driven approaches, where machine learning models learn patterns from labelled datasets. For example, SySeVR, introduced by Li et al. (Li et al. 2022), proposes a framework leveraging deep learning to analyse code slices and capture vulnerability-related patterns. The framework integrates tokenization, slicing, and graph-based representations to achieve state-of-the-art performance in detecting vulnerabilities within projects.

The integration of deep learning models has revolutionized vulnerability detection by enabling models to automatically extract meaningful representations from raw source code. Transformer-based models, such as BERT(Devlin et al. 2019) and its variants tailored for code such as CodeBERT (Feng et al. 2020), have demonstrated exceptional performance. These models leverage pre-training on large-scale code datasets, capturing nuanced language semantics that traditional approaches often miss (Ding et al. 2024).

However, recent studies have raised concerns about the real-world robustness of such models. Ding et al. (Zhu et al. 2022) introduced the PRIMEVUL benchmark and showed that state-of-the-art code language models, including GPT-4, perform close to random when evaluated on realistic, deduplicated, and chronologically split datasets. Zhu et al. (Zhu et al. 2025) surveyed the use of large language models in software security tasks such as fuzzing, program repair, and vulnerability analysis, highlighting their potential alongside critical shortcomings in semantic reasoning and contextual comprehension. Zhou et al. (Zhou et al. 2025)examined the security risks associated with deploying large language models, identifying threats such as prompt injection, data leakage, and adversarial misuse, and emphasised the need for robust mitigation strategies. Zhu et al. (Zhu et al. 2022) provided a comprehensive survey of fuzzing techniques, outlining generation- and mutation-based approaches and emphasising persistent challenges in seed selection, input modelling, and execution scalability. Complementing these works, Feng et al. (Feng et al. 2023) investigated vulnerability detection in IoT device firmware, identifying challenges related to firmware extraction, emulation, and tool automation, many of which remain unresolved in existing code-focused detection systems.

Methods like VULREM (Gürfidan 2024) and VulBERTa (Hanif and Maffeis 2022) fine-tune pre-trained models on project-specific data, enabling high precision and recall for within-project vulnerability detection. Compared to traditional approaches, transformer models offer superior adaptability and scalability, though their reliance on large pre-training datasets and high computational resources may present challenges.

Graph-based learning methods have also made significant contributions to vulnerability detection, representing source code as Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), or Data Flow Graphs (DFGs). These graphs capture structural and contextual relationships within the code, providing rich input for neural networks. For instance, frameworks like DiverseVul (Chen et al. 2023) combine graph-based representations with language models, enhancing the model's ability to detect vulnerabilities while retaining the project's structural integrity (Ma et al. 2022). Compared to transformer models, graph-based approaches excel in encoding structural patterns, such as data and control flows, which are often critical for identifying vulnerabilities in low-level programming languages like C or C++. However, graph-based methods may struggle to capture semantic nuances, making them less effective for projects where vulnerabilities stem from subtle logical errors or contextual dependencies.

Emerging technologies, such as quantum natural language processing (QNLP), further exemplify innovation in this domain. Akter et al. (Akter et al. 2023) explore the application of QNLP for automated vulnerability detection, demonstrating the potential of quantum computing to enhance the efficiency and accuracy of analysing complex code structures. While still in its infancy, QNLP represents a promising direction for future research, particularly in the context of highly intricate or large-scale systems.

Static analysis remains a cornerstone of vulnerability detection, particularly when augmented with ML techniques. For example, the work by Marjanov et al. (Marjanov et al. 2022) examines the current state of machine learning applications in source code vulnerability detection, identifying effective strategies and areas that require further development. While static analysis excels in identifying known patterns and vulnerabilities with deterministic signatures, its integration with ML models enables broader detection capabilities, particularly for previously unseen vulnerability patterns.

While within-project vulnerability detection has shown promising results in identifying vulnerabilities within a single project, it faces significant limitations when applied to real-world scenarios (Nguyen et al. 2020). The primary drawback is its reliance on project-specific labelled data, which may not always be available or sufficient for training robust models (Li et al. 2023a, b). Additionally, models trained on one project often fail to generalise effectively to other projects due to differences in coding styles, library dependencies, and structural patterns. This lack of cross-project generalisation limits the practical applicability of within-project methods, highlighting the need for more advanced approaches capable of detecting vulnerabilities across diverse software projects without requiring labelled data for each new target project.

2.2 Cross-Project vulnerability detection

Cross-Project Vulnerability Detection (CPVD) extends the capabilities of within-project approaches by addressing the challenges posed by domain discrepancies in software security. Unlike within-project detection, which relies on homogeneous datasets, CPVD focuses on generalising detection models to work across diverse software environments (Zhang et al. 2023). These environments often exhibit significant variability in coding practices, project-specific patterns, and library usage, requiring models to overcome domain shifts to perform effectively on unseen codebases. CPVD is particularly valuable in real-world sce-

narios where labelled data for target projects is scarce or unavailable, making the development of domain-agnostic solutions a critical research focus (Zhang et al. 2023).

Representation learning has played a pivotal role in CPVD methodologies by enabling models to extract transferable knowledge from source projects and apply it to target domains. For instance, Dam et al. (Lin et al. 2018) proposed a transfer learning framework that encodes universal patterns from vulnerable and non-vulnerable code into a shared representation space. Their approach demonstrated the feasibility of generalising vulnerability detection to unseen projects with minimal reliance on labelled data, emphasising the importance of domain-agnostic representations for effective cross-project adaptation.

Graph-based Neural Networks (GNNs) have emerged as powerful tools for CPVD due to their ability to model both structural and semantic relationships in source code. Zhang et al.(Zhang et al. 2023) introduced a framework that integrates Graph Attention Networks (GATs) with domain adaptation techniques, achieving feature distribution alignment between source and target projects. Their approach effectively captures the structural intricacies of code while mitigating the impact of domain-specific biases. Similarly, Wang et al. (Li et al. 2023a, b) employed graph embeddings to encode Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and Data Flow Graphs (DFGs), ensuring that syntactic and semantic features essential for vulnerability detection are preserved across domains. Their results highlight the utility of graph embeddings for achieving superior cross-project generalisation.

Domain adaptation techniques have been central to addressing domain shifts in CPVD. Nguyen et al. (Nguyen et al. 2020) proposed a dual-component framework that leverages adversarial learning and feature alignment to minimise domain divergence while preserving vulnerability-specific features. By aligning shared features across domains, their approach enhances the model's ability to detect vulnerabilities in diverse projects. Chen et al. (Liu et al. 2022) extended this concept with the CD-VulD framework, incorporating deep domain adaptation layers to align latent feature spaces and integrate structural and contextual features. These techniques underscore the importance of domain adaptation in CPVD and its role in improving model robustness.

Recent efforts have also explored hybrid models that combine multiple learning paradigms to enhance CPVD performance (Zhang et al. 2023). By integrating static analysis, graph-based learning, and representation learning, hybrid approaches address challenges such as class imbalance and variability in code structures. Additionally, emerging techniques like self-supervised and contrastive learning have demonstrated promise in pre-training models on large, unlabelled datasets, improving their ability to generalise and reducing false positives.

2.3 Research gaps and our contributions

Despite significant progress, CPVD still faces several key challenges. One major issue is aligning feature distributions between the source and target domains, especially when there are large differences in coding styles, structures, and libraries used across projects. Many existing domain adaptation methods struggle to handle the noise caused by inaccurate pseudo-labels, which can reduce the model's ability to correctly identify vulnerabilities. Additionally, most approaches rely solely on either graph-based models or transformer-based models, missing the potential benefits of combining both. Finally, class imbalance

and the lack of labelled data in the target domain remain major obstacles to developing reliable and scalable CPVD solutions for real-world use.

Our approach addresses these gaps by introducing a unified framework that integrates domain-agnostic CodeBERT embeddings, iterative pseudo-labelling, and ensemble refinement to improve cross-project generalisation. By aligning source and target feature distributions through iterative adaptation, the proposed method reduces domain discrepancies and ensures that vulnerability-relevant features are retained. Unlike existing techniques, our framework combines transformer-derived embeddings, leveraging their complementary strengths to enhance recall and minimise false negatives. This comprehensive strategy aims to overcome the limitations of existing CPVD methods and provide a robust, scalable solution for real-world software security challenges.

3 Methodology

This section presents the methodology behind ZSVulD, our proposed framework for zero-shot, cross-project vulnerability detection. The overall workflow is illustrated in Figs. 1 and 2. ZSVulD is designed to generalise across projects without requiring any labelled data from the target domain. It does so by combining frozen CodeBERT embeddings, iterative pseudo-labelling using a neural network, and final ensemble prediction using XGBoost.

The process begins with feature extraction. Code functions from both the source and target datasets are embedded using a pre-trained CodeBERT model, which is used without any fine-tuning. For each function, we compute the mean of its token embeddings to obtain

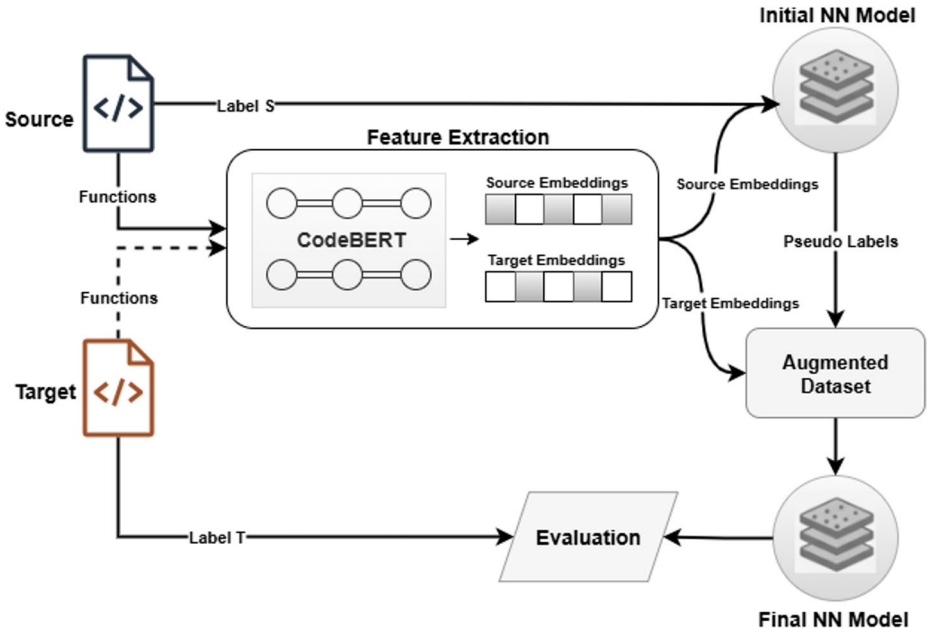

Fig. 1 The framework of the proposed ZSVulD approach. Label S represents the source dataset labels and Label T represents the target dataset labels

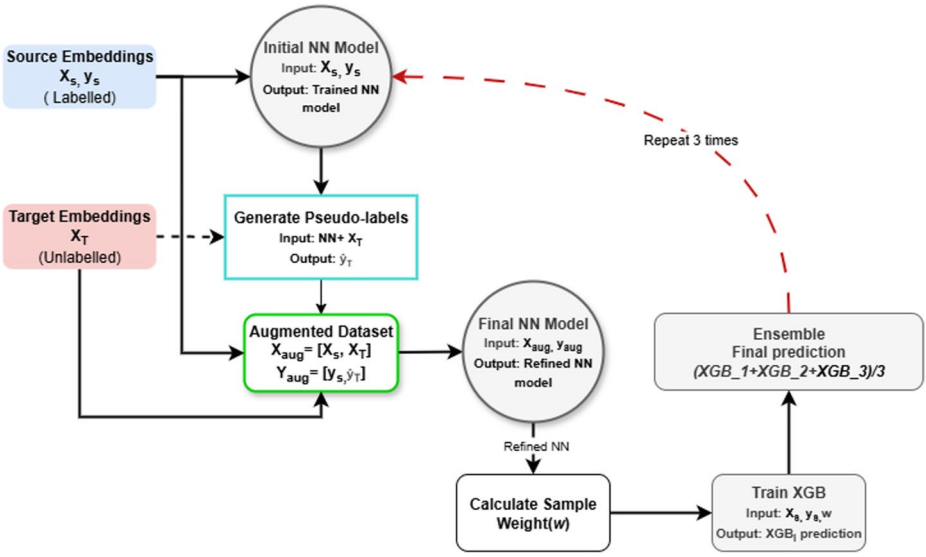

Fig. 2 The overview of the iterative pseudo-labelling and ensembling method

a fixed-length vector that captures both syntactic and semantic properties of the code. These embeddings are normalised to ensure consistent scaling across domains.

Next, a neural network is trained on the labelled source embeddings. This model is then used to generate pseudo-labels for the unlabelled target data. The source and pseudo-labelled target embeddings are combined to form an augmented dataset, which is used to retrain the neural network. This process is repeated three times to iteratively refine the pseudo-labels and improve adaptation to the target domain.

Following the final iteration, confidence scores from the neural model are used to compute sample weights, which indicate the reliability of each training instance. These weights, along with the source dataset, are used to train an XGBoost classifier. This procedure is carried out for each iteration, resulting in three separate XGBoost models. Their predictions are averaged to form the final output, allowing the ensemble to smooth out noise and stabilise predictions across domains.

To support full transparency and reproducibility, we have released the implementation at: https://github.com/Radowan98/ZSVulD.

The following subsections provide a detailed explanation of each component, outlining their specific roles and contributions within ZSVulD.

3.1 Problem formalisation

The task of zero-shot, cross-project vulnerability detection involves predicting the vulnerability labels of code functions in an unlabelled target dataset (D_T) using knowledge derived from a labelled source dataset (D_S). This section formalises the problem, identifies the inherent challenges, and establishes the constraints under which the proposed framework operates.

3.1.1 Notations and definitions

Let $D_S = \left(x_s^{(i)}, y_s^{(i)}\right) \mid i = 1, \ldots, N_S$represent the labelled source dataset, where $x_s^{(i)}$ denotes a function-level code snippet and $y_s^{(i)} \in 0{,}1$ is its corresponding vulnerability label. The target dataset, denoted as $D_T = x_t^{(j)} \mid j = 1, \ldots, N_T$, consists of unlabelled code snippets. The objective is to learn a classifier f that assigns vulnerability labels to the target dataset:

$$\widehat{y_t^{(j)}} = f\left(x_t^{(j)}\right) \tag{1}$$

3.1.2 Constraints and challenges

- Zero-Shot Setting: The classifier must generalise to unseen projects without labelled target data.
- Domain Shift: Feature distributions between the source and target datasets differ significantly.
- Class Imbalance: Vulnerable functions are often underrepresented, affecting classifier performance.
- Pseudo-Labelling Noise: Pseudo-labelling can introduce errors that propagate across training iterations without labelled target data.

3.2 Code representation using CodeBERT

To enable learning across source D_S and target D_T datasets, each code function is mapped into a fixed-length vector using CodeBERT, a transformer-based model pre-trained on programming and natural language corpora (Feng et al. 2020). CodeBERT encodes both syntactic and semantic features, making it suitable for representing source code in a domain-agnostic manner. Each function is first tokenised into a sequence of subword tokens. As the model accepts a maximum input length of 512 tokens, longer functions $(L > 512)$ are partitioned into $C = \frac{L}{512}$ non-overlapping chunks. Each chunk is processed independently by the frozen CodeBERT model to produce contextual token embeddings. For shorter functions (L ≤ 512), a single pass suffices, with padding applied where necessary.

To produce a single vector representation per function, token embeddings from all chunks are aggregated using mean pooling. This operation averages across all tokens and all chunks to yield a fixed-dimensional vector $h_{\text{mean}}(x) \in \mathbb{R}^{786}$, regardless of function length:

$$h_{\text{mean}}(x) = \frac{1}{C \times 512} \sum_{\text{l}=1}^{C} \sum_{i=1}^{512} h_i^{\text{l}}, \tag{2}$$

where h_i^{l} denotes the embedding of the i-th token in the l-th chunk. This representation preserves both local and global characteristics of the function while ensuring consistent input dimensions for downstream learning. The resulting embeddings are organised into

matrices $X_S \in \mathbb{R}^{N_S \times d}$ and $X_T \in \mathbb{R}^{N_T \times d}$ for the source and target datasets, respectively, where each row corresponds to a function embedding and $d = 768$. These matrices serve as input features to subsequent stages of the framework. By encoding functions from both domains in a shared vector space, the use of CodeBERT reduces the distributional gap caused by differences in coding style, structure, and project-specific patterns. The model is kept frozen throughout to preserve general-purpose knowledge acquired during pre-training (Devlin et al. 2019; Feng et al. 2020), thereby supporting transferability across projects.

3.3 Data preprocessing

Before training, the embeddings produced by CodeBERT for the source and target datasets are preprocessed to ensure feature compatibility and to address known issues such as distribution shift and class imbalance. These steps improve the robustness of cross-domain learning and help the model generalise beyond the training data.

Embeddings from different codebases may vary in distribution due to factors such as project-specific naming conventions, library dependencies, or function complexity. To reduce the impact of such variability, z-score normalisation is applied to each embedding dimension. This transforms each feature to have zero mean and unit variance across the dataset, thereby ensuring that features contribute on a comparable scale during optimisation. Normalisation is applied independently to both the source and target embeddings.

In addition to the distributional shift, class imbalance presents a further challenge. Vulnerability datasets typically contain a disproportionately large number of non-vulnerable functions compared to vulnerable ones. This imbalance can lead to skewed decision boundaries, with the model favouring the majority class and failing to identify vulnerable instances accurately. To mitigate this, class weights are computed for the source dataset using:

$$w_c = \frac{N_S}{2 \cdot N_c}, \quad c \in \{0,1\} \tag{3}$$

where N_S is the total number of source samples and N_c is the number of samples in class c. These weights assign greater importance to the minority class during training. The model is trained using a weighted binary cross-entropy loss function:

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{N_S} \sum_{i=1}^{N_S} w_{y_s^{(i)}} \left[y_s^{(i)} \log p_s^{(i)} + \left(1 - y_s^{(i)}\right) \log \left(1 - p_s^{(i)}\right) \right] \tag{4}$$

where $y_s^{(i)} \in \{0,1\}$ is the ground truth label for the i-th source function, and $p_s^{(i)}$ is the predicted probability of vulnerability. This weighting strategy reduces the bias toward the majority class and improves the model's sensitivity to vulnerable samples.

3.4 Iterative Pseudo-Labelling and neural network training

To reduce the domain gap between the source and target datasets, the framework employs an iterative pseudo-labelling procedure. This approach allows the model to gradually adapt to the target distribution in the absence of ground-truth labels. A neural network is first trained on the labelled source data and then used to generate pseudo-labels for the target functions. These pseudo-labelled samples are combined with the source data to create an augmented

training set. The model is then retrained on this set, and the process is repeated over multiple iterations. This iterative adaptation loop is illustrated in Fig. 2, which outlines the full training and refinement process. Iterative pseudo-labelling was selected over alternatives such as one-shot self-training (Li et al. 2023b) or co-training (Rothenberger and Diochnos 2023) for several reasons. Unlike one-shot self-training, which relies heavily on the initial pseudo-labels and risks overfitting to early errors, the iterative approach allows the model to progressively refine its understanding of the target domain. Compared to co-training, which requires two conditionally independent views of the data and can be difficult to realise with source code, iterative pseudo-labelling is simpler to implement and better suited to the single-view embedding space produced by pre-trained language models like CodeBERT. The gradual integration of pseudo-labelled samples provides a practical and effective means of domain adaptation under the constraints of zero-shot learning. The initial training stage involves fitting a neural network f_{NN} to the source embeddings $X_S \in R^{N_S \times d}$, where N_S is the number of labelled source samples and d is the embedding dimension. The model predicts binary vulnerability labels $y_s^{(i)} \in \{0,1\}$ for each function. Training is guided by the weighted binary cross-entropy loss:

$$\mathcal{L}_{\mathrm{weighted}} = -\frac{1}{N_S}\sum_{i=1}^{N_S} w_{y_s^{(i)}} \left[y_s^{(i)} \log p_s^{(i)} + \left(1 - y_s^{(i)}\right) \log\left(1 - p_s^{(i)}\right) \right] \tag{5}$$

where $p_s^{(i)} = f_{\mathrm{NN}}\left(x_s^{(i)}\right)$ is the model's predicted probability of vulnerability for the i-th source function, $w_{y_s^{(i)}}$ is the class weight defined earlier to account for label imbalance.

Once trained, the neural network is used to infer probabilities $p_t^{(j)} = f_{\mathrm{NN}}\left(x_t^{(j)}\right)$ for each function $x_t^{(j)}$ in the target dataset D_T. These probabilities are thresholded at a fixed value $\tau \in [0,1]$ (typically $\tau = 0.5$) to produce binary pseudo-labels (Haque et al. 2024):

$$\widehat{y_t^{(j)}} = \begin{cases} 1 & if\ p_t^{(j)} \geq \tau \\ 0 & otherwise\,. \end{cases}, \tag{6}$$

The pseudo-labelled target set $\left(X_T, \widehat{Y_T}\right)$ is then merged with the original source data to form an augmented training set:

$$D_{augmented} = \left(\boldsymbol{X}_S \cup \boldsymbol{X}_T, \boldsymbol{Y}_S \cup \widehat{\boldsymbol{Y}_T}\right), \tag{7}$$

The model is re-trained on this augmented dataset using the same weighted loss $\widehat{y_t^{(j)}}(i)$. After each iteration, the model produces updated probabilities for the target functions, which are re-thresholded to generate new pseudo-labels. The process is repeated for N iterations, with the.

model progressively refining its predictions as it is exposed to both labelled and pseudo-labelled examples.

Although pseudo-labelling introduces noise, particularly in early iterations, the framework incorporates mechanisms to limit its adverse effects. The ground-truth source labels carry higher confidence, ensuring that the model remains anchored during the adaptation process. In later stages, ensemble techniques are introduced to stabilise the quality of pseudo-labels across iterations. This iterative training strategy provides a foundation for

adapting the model to the target domain, using unlabeled data while maintaining alignment with the source distribution.

3.5 XGBoost refinement and ensemble learning

Although iterative pseudo-labelling allows the neural network to adapt gradually to the target domain, the absence of ground-truth labels and the potential for distributional mismatch can result in noisy predictions, particularly during the early stages. To improve robustness, the framework incorporates a second model, XGBoost (Chen and Guestrin 2016), which leverages the pseudo-labels and confidence scores generated by the neural network to stabilise learning and improve generalisation through ensemble averaging.

XGBoost is a gradient-boosted decision tree algorithm capable of capturing non-linear feature interactions that may be overlooked by deep neural models. In this framework, XGBoost is trained after each pseudo-labelling iteration using the augmented dataset:

$$D_{\text{aug}}^{(k)} = \left(\boldsymbol{X}_S \cup \boldsymbol{X}_T, \boldsymbol{Y}_S \cup \widehat{\mathrm{Y}}_{\mathrm{T}}^{(\mathrm{k})} \right) \tag{8}$$

where $\widehat{\mathrm{Y}}_{\mathrm{T}}^{(k)} = \left[\widehat{y}_t^{(1)}\left(k\right), \widehat{y}_t^{(2)}(k), \ldots ., \widehat{y}_t^{(N_T)}(k) \right]$ denotes the pseudo-labels for the target dataset generated in iteration k. Since pseudo-labels may contain noise, the framework introduces sample-level weights to indicate prediction confidence and modulate each sample's influence during XGBoost training.

For each target function $x_t^{(j)}$ the weight is computed as:

$$w_{\text{sample}}^{(j)} = \begin{cases} p_t^{(j)}(k) & if\ \widehat{y}_t^{(j)}(k) = 1, \\ 1 - p_t^{(j)}(k) & if\ \widehat{y}_t^{(j)}(k) = 0. \end{cases} \tag{9}$$

where $p_t^{(j)}(k) = f_{\text{NN}}\left(x_t^{(j)}\right)$ is the neural network's predicted probability for function $x_t^{(j)}$ at iteration k.

Using this information, the XGBoost model for iteration k is trained on the pseudo-labelled and weighted dataset:

$$f_{XGB}^{(k)} \leftarrow arg\ \min_f \sum_{i=1}^{n_s} w_{y_i^s} \cdot l\left(f\left(x_i^s\right),\ y_i^s\right) + \Omega\left(f\right) \tag{10}$$

Once trained, the model produces a prediction for each target function:

$$\widehat{\mathrm{y}}_{\mathrm{t}}^{(j)}(k) = f_{\mathrm{XGB}}^{(k)}\left(x_t^{(j)}\right) \tag{11}$$

After N iterations, the final ensemble score for each target function is computed by averaging the XGBoost predictions:

$$\text{Ensemble}_t^{(j)} = \frac{1}{N}\sum_{k=1}^{N} \widehat{\text{y}}_{\text{t}}^{(i)}(k) \tag{12}$$

A binary label is then assigned by thresholding the ensemble score at 0.5:

$$\widehat{y_t^{(j)}} = \begin{cases} 1 \;\; if \;\; Ensemble_t^{(j)} > 0.5, \\ 0 \qquad\qquad\qquad otherwise. \end{cases} \tag{13}$$

This ensemble-based procedure mitigates the effect of noisy pseudo-labels in early iterations. By averaging XGBoost predictions across multiple rounds each trained on pseudo-labelled data with confidence weights derived from the neural network, the framework improves robustness and generalisation to unseen target code. The final output is the set of predicted labels for the entire target dataset:

$$\widehat{\boldsymbol{Y}_T} = \left[\widehat{y_t^{(1)}}, \widehat{y_t^{(2)}}, \widehat{y_t^{(N_T)}}\right] \tag{14}$$

These predictions represent the model's estimate of function-level vulnerabilities in the target domain.

4 Experimental setup

The experimental setup provides details of the research questions, datasets, evaluation metrics, and implementation specifics used to validate the proposed framework for zero-shot, cross-project vulnerability detection.

4.1 Research questions

To evaluate the proposed ZSVulD approach, we have designed the following research questions (RQs):

RQ1: What is the impact of iterative pseudo-labelling on detection performance and stability in the absence of labelled target data?
RQ2: How does the ensemble mechanism contribute to prediction robustness in the zero-shot setting?
RQ3: How well does ZSVulD align source and target feature distributions, and what are the implications for cross-project generalisation?
RQ4: How effective is ZSVulD in detecting vulnerabilities across unseen projects in a zero-shot setting, compared to state-of-the-art cross-project methods?
RQ5: What are the computational requirements and limitations of ZSVulD in real-world cross-project scenarios?

4.2 Datasets

The experiments utilise two widely recognised datasets, Devign (Zhou et al. n.d.) and REVEAL (Chakraborty et al. 2022), which consists of labelled C code functions from a range of software projects. Both datasets include examples of vulnerable and non-vulnerable functions, making them suitable for training and evaluating vulnerability detection models. The combined dataset comprises a total of 30,260 code samples, with 14,700 labelled as vulnerable and 15,560 as non-vulnerable. Devign focuses on real-world vulnerability scenarios extracted from large-scale open-source projects such as FFmpeg and Qemu, providing a diverse set of challenging examples with varying coding styles and complexities. REVEAL, on the other hand, includes data from projects such as Debian and Chrome, offering additional diversity in terms of library usage and structural intricacies. These project-specific variations ensure that the datasets represent realistic and heterogeneous environments, ideal for assessing the cross-project generalisation capability of the proposed framework. For a fair comparison, we used the same dataset from the paper (Li et al. 2023a, b).

Further details and statistical breakdowns for these datasets are provided in Table 1, illustrating their scale and the distribution of vulnerable and non-vulnerable samples across projects.

4.3 Experimental details

All experiments were conducted using pre-extracted mean-pooled embeddings from a frozen CodeBERT model, normalised using z-score standardisation. UniXcoder was also included as a baseline and fine-tuned for classification on the same source datasets. Fine-tuning of CodeBERT and UniXcoder was performed using the Adam optimiser with a learning rate of 1×10^{-5}. Models were trained using a 70:15:15 split for training, validation, and testing, with early stopping based on validation F1 score and a patience of 2 epochs. Maximum training duration was capped at 10 epochs. The neural network used in ZSVulD consists of four hidden layers with ReLU activations and dropout (rate 0.2). Weighted binary cross-entropy loss was applied to address class imbalance, and learning was carried out using Adam with the same learning rate as above. For the XGBoost component, we configured XGBoost with 100 trees and a maximum depth of 3, which is a common and effective choice for limiting overfitting and ensuring a fast training process. This configuration balances predictive capacity with regularisation, especially important when training on pseudo-labelled data, which may contain noise.Ensemble outputs were computed by averaging predictions over three XGBoost models, each trained using data from successive pseudo-labelling iterations.

All experiments were run on a workstation equipped with an Intel Core i9-11900 K CPU, 64 GB RAM, and an NVIDIA RTX 4090 GPU.

Table 1 Experimental dataset descriptions

Project	Dataset	Vulnerable	Non-Vulnerable	Total
Devign	FFmpeg	4981	5016	9997
	Qemu	7479	7980	15,459
REVEAL	Debian	1415	1653	3068
	Chrome	825	911	1768
Total		14,700	15,560	30,260

4.4 Comparative analysis and baseline models

For comparative analysis, the performance of the proposed framework was evaluated alongside existing methods from the literature, including VulGDA (Li et al. 2023a, b), VulDPecker (Li et al. 2018), and SCDAN (Li et al. 2022). The results for these baseline methods we obtained from this paper (Li et al. 2023a, b), which reported their performance on the same datasets used in this study. This approach ensures a fair and consistent comparison under identical experimental settings.

To include recent language model-based techniques, we additionally evaluate UniXcoder(Guo et al. 2022) as a representative LLM baseline. UniXcoder supports fine-tuning and is designed for source code tasks, making it suitable for supervised zero-shot evaluation. In contrast, decoder-only models such as GPT require prompt-based inference and are not directly compatible with classification pipelines without extensive adaptation, which may compromise fairness in empirical comparison.

4.5 Evaluation metrics

To evaluate the proposed zero-shot, cross-project vulnerability detection framework, we report Precision, Recall, F1-score, and F2-score. These metrics are standard for binary classification tasks with class imbalance, where the vulnerable class is typically underrepresented. Recall is critical to ensure vulnerabilities are not missed, while precision reduces the burden of false positives on developers. F1 provides a balanced trade-off, and F2 gives greater weight to recall, reflecting the security-sensitive nature of vulnerability detection.

5 Results

This section reports the experimental findings of ZSVulD, structured around the five research questions introduced earlier.

5.1 Effect of iterative pseudo-labelling(RQ1)

To assess the impact of iterative pseudo-labelling in cross-project vulnerability detection, we conducted experiments across three transfer scenarios using a neural network trained initially on labelled source data. The training set was progressively augmented with predictions from the unlabelled target domain over two iterations. This process was designed to improve the model's alignment with the target distribution without access to any ground-truth labels. We report precision, recall, F1 score, and F2 score for each iteration in Table 2.

In the Devign to ReVeal setting, the first iteration of pseudo-labelling led to a marked increase in recall, rising from 0.7921 to 0.8742. F2 also improved from 0.6988 to 0.7509, indicating better sensitivity to vulnerable samples. Precision remained stable at approximately 0.485, while the F1 score increased from 0.5981 to 0.6219. The second iteration produced no further improvements, suggesting that the model had reached convergence. These results indicate that the model effectively adapted to the target domain after a single iteration, without evidence of overfitting or instability.

Table 2 Effect of iterative pseudo-labelling

Source → Target	Iteration	Precision	Recall	F1 Score	F2 Score
Devign → ReVeal	1	0.4876	0.7921	0.5981	0.6988
	2	0.4855	0.8742	0.6219	0.7509
	3	0.4852	0.8736	0.6219	0.7508
Qemu+ReVeal → FFmpeg	1	0.5105	0.7843	0.6089	0.7004
	2	0.5122	0.7576	0.5965	0.6811
	3	0.5111	0.7667	0.6097	0.6939
Devign+Chrome → Debian	1	0.5064	0.8299	0.6253	0.7322
	2	0.5170	0.8753	0.6484	0.7671
	3	0.5216	0.8630	0.6463	0.7590

In the Qemu+ReVeal to FFmpeg transfer, performance remained largely unchanged across iterations. Recall decreased slightly from 0.7843 to 0.7667, and F2 dropped from 0.7004 to 0.6939. Precision stayed consistent at around 0.511, while F1 showed minimal fluctuation between 0.6089 and 0.6097. This suggests that pseudo-labelling offered limited benefit in this configuration, likely due to a lower degree of similarity between the source and target domains. Nonetheless, the absence of performance degradation across iterations points to a stable learning process.

In the Devign+Chrome to Debian scenario, the first iteration yielded consistent improvements. Recall increased from 0.8299 to 0.8753, and F2 rose from 0.7322 to 0.7671. F1 improved from 0.6253 to 0.6484, and precision also increased slightly from 0.5064 to 0.5170. The second iteration provided only marginal additional gains, reinforcing the observation that pseudo-labelling is most effective during early adaptation. Performance remained stable across seeds and iterations, supporting the reliability of the observed improvements.

Across all three scenarios, the first iteration of pseudo-labelling consistently enhanced recall and F2 without negatively affecting precision. This demonstrates that iterative pseudo-labelling can improve detection performance in the absence of labelled target data, particularly in settings where the source and target domains share structural or semantic similarity. Moreover, performance across iterations remained stable, with no signs of overfitting or instability, even as the model incorporated pseudo-labelled samples. These findings indicate that a single iteration is often sufficient to achieve meaningful adaptation while maintaining reliable performance, offering an efficient and stable approach to zero-shot vulnerability detection.

5.2 Effect of ensembling on Zero-Shot Cross-Project Detection(RQ2)

To evaluate the effect of the ensemble mechanism on prediction robustness in the zero-shot setting, we compared three modelling strategies, each trained without access to labelled target data. The first used a neural network trained with source data augmented by pseudo-labelled target samples. The second applied XGBoost to the source data only, using prediction confidence from the neural network as instance weights to stabilise learning. The third created an ensemble by training multiple XGBoost models with weights from final NN model and aggregating their outputs via mean probability averaging.

We evaluated both mean and soft voting as ensemble aggregation strategies and found that they produced nearly identical results across all transfer settings. We therefore report

only the mean ensemble in Table 3, which offers practical benefits. Specifically, mean aggregation retains continuous confidence estimates, facilitating downstream thresholding and risk-sensitive decision-making. Based on these considerations, we adopt mean-based ensembling as our default approach.

In the Devign to ReVeal transfer scenario, the ensemble achieved an F2 score of 0.77, outperforming the neural network (0.7431) and the XGBoost model (0.7420). Recall increased to 0.87, while precision rose modestly to 0.52, resulting in an F1 score of 0.65. This reflects improved coverage of vulnerable samples without sacrificing calibration. The ensemble demonstrated stronger recall than either base model, with slightly improved F1 and F2, indicating a balanced and robust output.

In the Qemu+ReVeal to FFmpeg setting, which involves greater domain divergence, the ensemble again outperformed both baselines. It achieved an F2 score of 0.69 and F1 of 0.63, with a recall of 0.73 and precision of 0.53. These improvements, though moderate, suggest that aggregation helped mitigate model instability in a more challenging zero-shot configuration.

In the Devign+Chrome to Debian transfer, the ensemble delivered the highest scores across all metrics. It achieved a recall of 0.85, precision of 0.54, F1 of 0.66, and F2 of 0.76. These results show that the ensemble effectively captured the high recall of the neural network and the relatively stronger precision of the XGBoost classifier, producing a more stable and balanced predictor.

Across all scenarios, the ensemble consistently matched or exceeded the performance of individual models in both F1 and F2, with no degradation in precision. The improvement in recall and robustness, particularly in low-resource and cross-domain settings, demonstrates that ensemble-based strategies enhance generalisability and stability when no labelled target data is available. These findings confirm the value of model ensembling as a reliable mechanism for robust detection in zero-shot vulnerability analysis.

5.3 Feature alignment and Cross-Project Generalisation(RQ3)

To assess the ability of ZSVulD to align feature distributions across domains, we analysed both qualitative and quantitative indicators of distributional alignment between source and target datasets. This included t-SNE visualisation of learned representations before and after adaptation, and measurement of domain discrepancy using Maximum Mean Discrepancy (MMD). Experi-

Table 3 Effect of ensembling on unseen dataset

Source → Target	Model	Precision	Recall	F1 Score	F2 Score
Devign → ReVeal	NN	0.5047	0.8437	0.6309	0.7431
	XGBoost	0.5019	0.8429	0.6291	0.7420
	Ensemble	0.5200	0.8700	0.6500	0.7700
Qemu+ReVeal → FFmpeg	NN	0.5280	0.5826	0.5446	0.5643
	XGBoost	0.5276	0.6251	0.5722	0.6028
	Ensemble	0.5300	0.7300	0.6300	0.6900
Devign+Chrome → Debian	NN	0.5324	0.8206	0.6436	0.7383
	XGBoost	0.5511	0.7053	0.6187	0.6679
	Ensemble	0.5400	0.8500	0.6600	0.7600

ments were conducted across three representative cross-project settings: Devign+Chrome to Debian (Fig. 3), Qemu+ReVeal to FFmpeg (Fig. 4), and Devign to ReVeal (Fig. 5).

Prior to adaptation, the t-SNE plots (Figs. 3 and 4, and 5*)* show that source and target samples are largely disjoint in the latent space, indicating a lack of meaningful alignment. At this stage, MMD values were low: 0.0005 for Debian, 0.0002 for FFmpeg, and 0.0003 for ReVeal. These low scores reflect the tightly clustered nature of raw CodeBERT embeddings, which tend to preserve syntactic similarity rather than vulnerability-specific structure. Although low MMD is often associated with alignment, in this case it signals under-expressiveness in the pre-trained embedding space, which limits the model's ability to discriminate across domains.

Following adaptation via iterative pseudo-labelling and ensemble refinement, the latent space becomes more discriminative. The t-SNE plots (Figs. 4 and 5, and 5) reveal increased overlap between source and target distributions, indicating that the model has learned a more generalisable representation. This qualitative shift is matched by substantial increases in MMD values: 0.1525 for Debian, 0.0210 for FFmpeg, and 0.0866 for ReVeal. These increases reflect a move to a higher-variance, task-relevant embedding space that better captures vulnerability semantics. While MMD is conventionally interpreted as a distance metric, the relative increase in this context suggests greater expressiveness and task-specific separation in the adapted representations.

As shown in *Table X*, the relative increase in MMD was most pronounced for Debian (+29,741.36%), followed by ReVeal (+26,059.06%) and FFmpeg (+12,221.08%). This

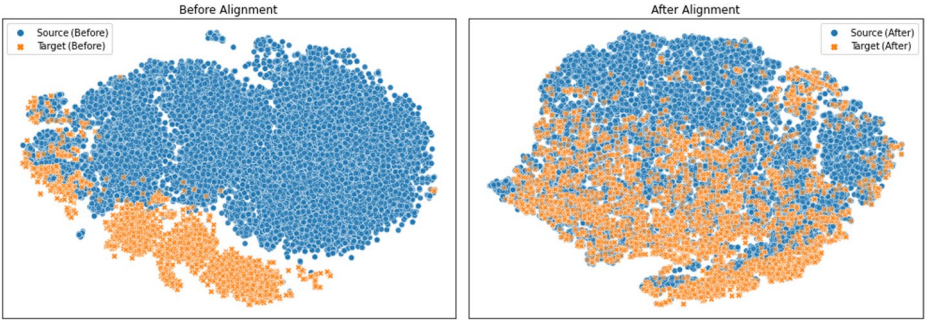

Fig. 3 Feature Alignment: Devign → Reveal

Fig. 4 Feature Alignment: Reveal+Qemu → FFmpeg

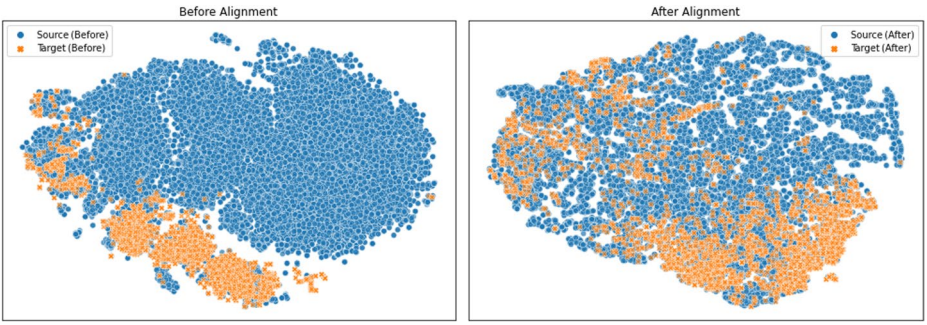

Fig. 5 Feature Alignment: Devign+Chrome → Debian

indicates that ZSVulD achieves the strongest alignment in settings with heterogeneous source domains, such as Debian, where diverse training data may enhance generalisability. In contrast, the smaller shift observed in FFmpeg suggests persistent distributional divergence, consistent with earlier findings in RQ1 and RQ2. Nonetheless, even in that scenario, alignment improved substantially relative to the initial embedding space.

Taken together, the t-SNE visualisations and MMD scores (Figs. 3, 4, and 5; Table 4) demonstrate that ZSVulD improves cross-domain feature alignment by inducing a task-aware latent space that captures vulnerability-relevant distinctions. This transformation enables better generalisation to unseen projects, even in the absence of labelled target data. By bridging the domain gap through unsupervised adaptation, ZSVulD mitigates the adverse effects of distribution shift, providing a robust foundation for zero-shot vulnerability detection.

5.4 Comparative Effectiveness(RQ4)

To evaluate the effectiveness of ZSVulD in detecting vulnerabilities across unseen projects, we conducted a comparative analysis against several state-of-the-art cross-project vulnerability detection methods. As shown in Table 5, the proposed framework was benchmarked against VulGDA (Li et al. 2023a, b), VulDPecker (Li et al. 2018), SCDAN (Li et al. 2022), and CodeBERT (Feng et al. 2020), and UniXcoder (Guo et al. 2022). The evaluation was carried out across three representative zero-shot transfer settings: Reveal+Qemu to FFmpeg, Devign to ReVeal, and Devign+Chrome to Debian.

In the Reveal+Qemu to FFmpeg setting, ZSVulD achieved a recall of 0.73, outperforming all baselines including VulGDA (0.61), VulDPecker (0.59), SCDAN (0.67), CodeBERT (0.66), and UniXcoder (0.69). The corresponding F1 score (0.63) and F2 score (0.69) were also higher or comparable, indicating improved sensitivity while maintaining calibration.

In the Devign to ReVeal setting, ZSVulD achieved a recall of 0.87, substantially higher than VulGDA (0.52), SCDAN (0.62), and CodeBERT (0.66). Although the precision was

Table 4 MMD shift before and after alignment

Source Domain	Target Domain	MMD (Before)	MMD (After)	Relative Increase
Devign	Debian	0.0005	0.1525	+29,741.36%
Reveal+Qemu	FFmpeg	0.0002	0.0210	+12,221.08%
Devign+Chrome	ReVeal	0.0003	0.0866	+26,059.06%

Table 5 The performance of the proposed framework compared to the state-of-the-art methods

Source	Target	Method	Recall	Precision	F1-score	F2-score
Reveal+Qemu	FFmpeg	VulGDA (Li et al. 2023a, b)	0.61	**0.65**	0.63	0.62
		VulDPecker (Li et al. 2018)	0.59	0.51	0.55	0.57
		SCDAN (Li et al. 2022)	0.67	0.60	0.63	0.65
		CodeBERT (Feng et al. 2020)	0.66	0.58	0.62	0.64
		UniXcoder (Guo et al. 2022)	0.69	0.56	0.64	0.69
		Proposed	**0.73**	0.53	**0.63**	**0.69**
Devign	Reveal	VulGDA (Li et al. 2023a, b)	0.52	**0.67**	0.59	0.54
		VulDPecker (Li et al. 2018)	0.61	0.54	0.57	0.59
		SCDAN (Li et al. 2022)	0.62	0.56	0.59	0.61
		CodeBERT (Feng et al. 2020)	0.66	0.54	0.59	0.63
		UniXcoder (Guo et al. 2022)	0.70	0.51	0.61	0.66
		Proposed	**0.87**	0.52	**0.65**	**0.77**
Devign+Chrome	Debian	VulGDA (Li et al. 2023a, b)	*N/A**	*N/A**	*N/A**	*N/A**
		VulDPecker (Li et al. 2018)	*N/A**	*N/A**	*N/A**	*N/A**
		SCDAN (Li et al. 2022)	*N/A**	*N/A**	*N/A**	*N/A**
		CodeBERT (Feng et al. 2020)	0.71	0.52	0.61	0.66
		UniXcoder (Guo et al. 2022)	0.66	0.50	0.58	0.61
		Proposed	**0.85**	**0.54**	**0.66**	**0.76**

**These settings are not available in their paper.*

slightly lower at 0.52, the F1 score (0.65) and F2 score (0.77) surpassed those of all other methods, including UniXcoder. This demonstrates the framework's ability to detect a high proportion of vulnerable samples, which is especially important in security-critical contexts where false negatives must be minimised.

In the Devign+Chrome to Debian setting, where only CodeBERT and UniXcoder results are available, ZSVulD again outperformed the baselines. It achieved a recall of 0.85, compared to 0.71 for CodeBERT and 0.66 for UniXcoder. The F1 and F2 scores were the highest in this setting, at 0.66 and 0.76 respectively. This confirms the model's capacity to generalise effectively across heterogeneous source domains.

Across all scenarios in Table 4, ZSVulD consistently exceeded the performance of existing cross-project detection methods, particularly in terms of recall and F2 score. These metrics are especially critical in vulnerability detection, where identifying true positives is often prioritised over suppressing false alarms. While ZSVulD's precision is slightly lower, ranging from 0.52 to 0.54, this trade-off is acceptable in security-focused applications where missed vulnerabilities can have severe consequences.

In summary, ZSVulD delivers strong detection performance in zero-shot settings without requiring target labels. Its consistent improvements in recall, F1, and F2 across multiple transfer configurations highlight its practical value for real-world vulnerability detection across unseen projects.

5.5 Computation cost Analysis(RQ5)

To assess the computational feasibility of ZSVulD in practical settings, we measured its inference time, embedding cost, and training duration across three representative cross-project

transfer configurations: Devign to ReVeal, Devign + Chrome to Debian, and Qemu + ReVeal to FFmpeg. The results are summarised in Tables 6 and 7.

As shown in Table 6, inference latency was consistently low across all targets. The XGBoost classifier achieved the fastest runtime, processing a single function in 0.00104 to 0.00163 milliseconds. The neural network, which contains 57,057 trainable parameters, required 0.02775 to 0.05629 milliseconds per function, depending on the configuration. Although the neural model is slower than XGBoost, both operated well under a one-millisecond threshold. This makes them suitable for scalable static analysis, including file-level or batch-mode evaluation in automated pipelines. The cost of generating function-level embeddings using the fine-tuned CodeBERT encoder was also measured. This step required approximately 203 milliseconds per function across all domains. As embedding is a one-time preprocessing operation, it does not affect online inference latency. Furthermore, this cost can be parallelised or cached during deployment, and amortised over multiple reuse scenarios or large-scale code repositories. The total training duration for ZSVulD includes multiple rounds of pseudo-labelling and model retraining. As reported in Table 7, neural network training time ranged from 169.86 to 183.12 s, while XGBoost training required only 6.48 to 7.53 s. The full training cycle remained below 3.2 min for all evaluated transfer scenarios. These figures suggest that ZSVulD can be retrained regularly and integrated into scheduled workflows, such as continuous integration or offline vulnerability triage. Across all settings, ZSVulD demonstrated low computational overhead, sub-millisecond inference latency, and modest training requirements. The only significant cost lies in the embedding stage, which is independent of downstream inference and can be optimised using standard parallelisation techniques. No GPU is required beyond the initial embedding phase, and the model sizes are small enough to support efficient CPU-based deployment. In summary, ZSVulD is computationally efficient and well suited for use in real-world cross-project vulnerability detection scenarios. Its lightweight architecture and low-latency inference support high-throughput batch processing, while its retraining time allows for integration into periodic or event-triggered update cycles. The framework is therefore feasible for practical deployment in security pipelines, particularly where automation and scalability are required.

Table 6 Inference time analysis

Source	Target	NN Inference Time (ms)	XGBoost Inference Time (ms)
Devign + Chrome	Debian	0.05629	0.00163
Devign	ReVeal	0.04286	0.00104
ReVeal + Qemu	FFmpeg	0.02775	0.00105

Table 7 Training time analysis

Source Domain	Target	NN Training Time (s)	XGBoost Training Time (s)	Total ZSVulD Time (s)
Devign + Chrome	Debian	183.12	7.53	190.64
Devign	ReVeal	182.12	7.08	189.20
ReVeal + Qemu	FFmpeg	169.86	6.48	176.34

6 Discussion

The experimental results indicate that ZSVulD offers a practical solution for zero-shot, cross-project vulnerability detection. Most performance improvements are achieved after the first round of pseudo-labelling (RQ1), with diminishing returns in subsequent iterations. The ensemble mechanism, which combines confidence-weighted XGBoost classifiers trained at each stage, contributes to prediction stability and maintains consistent recall across evaluation settings (RQ2). Adaptation improves alignment between source and target domains, as reflected in lower MMD values and more coherent clusters in the t-SNE visualisations (RQ3). ZSVulD achieves competitive recall and F2 scores across all scenarios, with moderate precision trade-offs (RQ4). The framework is also efficient in practice, with low training overhead and fast inference times. The most resource-intensive step is embedding generation using CodeBERT, which can be parallelised to improve throughput (RQ5).

6.1 Failure analysis

To examine the limitations of ZSVulD, we estimated the numbers of false positives (FP) and false negatives (FN) for three representative cross-project settings using the reported precision, recall, and the number of vulnerable samples in the target domain. This approach provides a more informative view of model behaviour than accuracy alone, particularly in the context of class imbalance. Given the design goal of prioritising vulnerability detection, reducing false negatives is treated as the primary criterion. As shown in Table 8, the highest number of false negatives (1,345) occurs in the Qemu + ReVeal to FFmpeg setting, which also has the highest number of false positives (3,225). This indicates that ZSVulD performs less effectively in scenarios with high structural variability and domain shift, which is consistent with the relatively lower precision and recall reported in this configuration. In contrast, the Devign to ReVeal and Devign + Chrome to Debian settings record substantially fewer false negatives, with 162 and 212 cases respectively. These lower FN counts are consistent with the higher recall observed in these settings, suggesting that the framework identifies vulnerable functions more reliably when there is greater overlap or diversity in the source domains.

The false positive counts are consistently high across all settings, ranging from 996 to 3,225, reflecting the framework's bias towards recall. In the ReVeal target, for example, 996 false positives were estimated, which is more than six times the number of false negatives (162). A similar pattern is observed in the Debian setting, where the FP count is almost five times higher than the FN count. This outcome reflects a design choice where sensitivity is prioritised over specificity, a trade-off that can be acceptable in security applications where the cost of missing a true vulnerability outweighs the cost of flagging non-vulnerable code. A further contributor to misclassification is the use of pseudo-labelled target samples. In the initial iterations, the neural

Table 8 False negatives and false positives across Cross-Project evaluation settings

Source Setting	Target	Vulnerable Samples	False Negatives (FN)	False Positives (FP)
Devign	ReVeal	2240	162	996
ReVeal + Qemu	FFmpeg	4,981	1,345	3,225
Devign + Chrome	Debian	1,415	212	1,025

network is trained solely on source data and may assign incorrect labels to structurally different target code. The noise introduced at this stage can propagate into the ensemble, particularly if early predictions are poorly calibrated. Although the XGBoost component and averaging across iterations reduce this effect, they do not fully remove errors arising from unreliable pseudo-labels. These limitations are most evident in the FFmpeg setting, where the highest FP and FN counts coincide with the lowest cross-project alignment and task performance.

7 Threats to validity

While the proposed framework demonstrates robust performance in zero-shot, cross-project vulnerability detection, several potential threats to validity must be considered. We organise these into internal, external, and construct validity.

7.1 Internal validity

Internal validity relates to whether the observed results can be reliably attributed to the proposed method.

A possible threat lies in the reliance on pre-trained embeddings from CodeBERT, which may introduce biases inherited from its training corpus. Since CodeBERT was trained on large-scale open-source code, it may favour commonly used coding conventions or patterns. If such patterns are unevenly distributed across the source and target projects, this could influence model performance. Another source of potential error arises from the pseudo-labelling process, particularly during early iterations. Although we apply sample weighting derived from refine NN model and ensemble refinement to mitigate label noise, the neural model's initial predictions may still be unreliable on unseen domains. These early-stage pseudo-labels can propagate through the ensemble stage, affecting final predictions. The framework intentionally prioritises recall and F2-score over accuracy, to reduce the risk of missing real vulnerabilities. This trade-off may lead to higher false positive rates, which, while acceptable in many security contexts, may not be ideal for all deployment scenarios. Additional work is needed to explore the practical impact of this balance in realistic settings.

7.2 External validity

External validity concerns the extent to which the findings generalise beyond the current experimental setup.

This study uses two widely cited datasets: Devign and ReVeal, both of which contain real-world C/C++ functions with binary vulnerability labels. These datasets offer meaningful diversity across open-source projects, but do not cover other programming languages or industrial codebases. The generalisability of the proposed approach to other languages (e.g. Java, Python) or software ecosystems has not yet been evaluated.

Furthermore, while the cross-project design simulates realistic deployment scenarios, the number of domains is limited. Projects with highly specialised coding styles, security practices, or domain-specific libraries may present challenges not captured in the current evaluation. Expanding to broader, multilingual, or proprietary datasets would strengthen the generalisability of our findings.

7.3 Construct validity

Construct validity addresses whether the experimental design and metrics accurately capture the intended concepts.

The task of vulnerability detection is framed as a binary classification problem using dataset-provided labels. These labels, while curated, may still include noise due to their derivation from commit messages or automated analysis. Such label imperfections can affect both model training and reported performance. Additionally, while recall, precision, F1, and F2 scores are appropriate for quantifying classification performance, they do not fully capture the real-world cost of errors. In practice, a missed vulnerability may have more serious consequences than a false alert. Although the use of F2-score reflects this asymmetry to some extent, more nuanced evaluation methods could provide further insight into the model's practical utility.

8 Conclusion and future work

This paper presents ZSVulD, a framework for zero-shot, cross-project vulnerability detection that addresses practical challenges in adapting to new software domains without labelled target data. The approach uses frozen CodeBERT embeddings to extract domain-agnostic representations of source code, capturing both syntactic and semantic information. These embeddings are processed through an iterative pseudo-labelling mechanism, where a neural network assigns and refines labels for target samples over multiple iterations. The framework then applies ensemble refinement using XGBoost, guided by sample weights, to improve prediction stability across domains. Empirical evaluation on two real-world datasets, Devign and ReVeal, shows that ZSVulD achieves strong recall and balanced F1 and F2 scores in several cross-project settings. These results suggest that the framework is well-suited to scenarios where recall is critical, such as vulnerability detection in CI/CD pipelines or large-scale open-source audits. Future work includes exploring the integration of Code Property Graph features to enrich code representations, as well as developing improved pseudo-labelling strategies that incorporate confidence-aware filtering or noise-tolerant mechanisms to reduce error propagation during adaptation.

Acknowledgements This research is supported by the ARC (Advanced Research Engineering Centre) project. PwC* is in receipt of Grant for R&D support from Invest NI for ARC. This project is part-financed by the European Regional Development Fund under the Investment for Growth and Jobs Programme 2014–2020. *PricewaterhouseCoopers LLP a limited liability partnership incorporated in England with its registered office at 1 Embankment Place, London WC2N 6RH.

Author contributions **Radowanul Haque**- Conceptualisation, analysis, and manuscript drafting.
Aftab Ali, Sally McClean, and naveed Khan – Supervision, critical feedback, and manuscript revision.

Funding This research did not receive any external funding.

Data availability To support full transparency and reproducibility, we have released the implementation at: https://github.com/Radowan98/ZSVulD.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

Akter MS, Shahriar H, Bhuiya ZA (2023) Automated vulnerability detection in source code using quantum natural Language processing. Commun Comput Inform Sci 1768 CCIS:83–102. https://doi.org/10.1007/978-981-99-0272-9_6

Cai Z, Cai Y, Chen X et al (2024) CSVD-TF: Cross-project software vulnerability detection with tradaboost by fusing expert metrics and semantic metrics. J Syst Softw 213:112038. https://doi.org/10.1016/j.jss.2024.112038

Chakraborty S, Krishna R, Ding Y, Ray B (2022) Deep learning based vulnerability detection: are we there yet? IEEE Trans Software Eng 48:3280–3296. https://doi.org/10.1109/TSE.2021.3087402

Chen T, Guestrin C (2016) XGBoost: A scalable tree boosting system. Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining 13-17-August-2016:785–794. https://doi.org/10.1145/2939672.2939785/SUPPL_FILE/KDD2016_CHEN_BOOSTING_SYSTEM_01-ACM.MP4

Chen Y, Ding Z, Alowain L et al (2023) DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection. ACM Int Conf Proceeding Ser 15:654–668. https://doi.org/10.1145/3607199.3607242

David A, Wheeler (2017) Flawfinder Home Page. https://dwheeler.com/flawfinder/. Accessed 18 Jan 2025

Devlin J, Chang M-W, Lee K et al (2019) BERT: Pre-training of deep bidirectional Transformers for Language Understanding. Proc 2019 Conf North 4171–4186. https://doi.org/10.18653/V1/N19-1423

Ding Y, Fu Y, Ibrahim O et al (2024) Vulnerability Detection with Code Language Models: How Far Are We? 1729–1741. https://doi.org/10.1109/icse55347.2025.00038

Feng Z, Guo D, Tang D et al (2020) CodeBERT: A Pre-Trained model for programming and natural languages. Findings of the association for computational linguistics: EMNLP 2020. Association for Computational Linguistics, Stroudsburg, PA, USA, pp 1536–1547

Feng X, Zhu X, Han QL (2023) Detecting vulnerability on IoT device firmware: a survey. IEEE/CAA Journal of Automatica Sinica 10:25–41. https://doi.org/10.1109/JAS.2022.105860

Guo D, Lu S, Duan N et al (2022) UniXcoder: unified Cross-Modal Pre-training for code representation. Proc Annual Meeting Association Comput Linguistics 1:7212–7225. https://doi.org/10.18653/v1/2022.acl-long.499

Gürfidan R (2024) Vulrem: Fine-tuned BERT-based source-code potential vulnerability scanning system to mitigate attacks in web applications. Appl Sci 14:9697. https://doi.org/10.3390/APP14219697

Hanif H, Maffeis S (2022) VulBERTa: Simplified Source Code Pre-Training for Vulnerability Detection. Proceedings of the International Joint Conference on Neural Networks 2022-July: https://doi.org/10.1109/IJCNN55064.2022.9892280

Haque R, Ali A, Mcclean S et al (2024) Heterogeneous cross-project defect prediction using encoder networks and transfer learning. IEEE Access 12:409–419. https://doi.org/10.1109/ACCESS.2023.3343329

Harzevili NS, Belle AB, Wang J et al (2023) A survey on automated software vulnerability detection using machine learning and deep learning. ACM Comput Surv 37. https://doi.org/10.1145/3699711/ASSET/CF5E99B1-65E5-4D5E-B788-FFDFB5725C05/ASSETS/GRAPHIC/CSUR-2023-0542-U07.JPG

IBM (2024) Cost of a data breach 2024 | IBM. https://www.ibm.com/reports/data-breach. Accessed 29 Dec 2024

Kaur A, Nayyar R (2020) A comparative study of static code analysis tools for vulnerability detection in C/C++and JAVA source code. Procedia Comput Sci 171:2023–2029. https://doi.org/10.1016/j.procs.2020.04.217

Li Z, Zou D, Xu S et al (2018) VulDeePecker: A deep Learning-Based system for vulnerability detection. 25th Annual Netw Distrib Syst Secur Symp NDSS 2018. https://doi.org/10.14722/ndss.2018.23158

Li Z, Zou D, Xu S et al (2022) Sysevr: a framework for using deep learning to detect software vulnerabilities. IEEE Trans Depend Secure Comput 19:2244–2258. https://doi.org/10.1109/TDSC.2021.3051525

Li X, Xin Y, Zhu H (2023a) Cross-domain vulnerability detection using graph embedding and domain adaptation. Computers & Secur 125:103017. https://doi.org/10.1016/J.COSE.2022.103017
Li Y, Hui B, Xia X et al (2023b) One-Shot Learning as Instruction Data Prospector for Large Language Models. Proceedings of the Annual Meeting of the Association for Computational Linguistics 1:4586–4601. https://doi.org/10.18653/v1/2024.acl-long.252
Lin G, Zhang J, Luo W et al (2018) Cross-project transfer representation learning for vulnerable function discovery. IEEE Trans Ind Inform 14:3289–3297. https://doi.org/10.1109/TII.2018.2821768
Lin G, Wen S, Han QL et al (2020) Software vulnerability detection using deep neural networks: a survey. Proc IEEE 108:1825–1848. https://doi.org/10.1109/JPROC.2020.2993293
Liu S, Lin G, Qu L et al (2022) CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation. IEEE Trans Depend Secure Comput 19:438–451. https://doi.org/10.1109/TDSC.2020.2984505
Ma M, Zhao Z, Soremekun E et al (2022) GraphCode2Vec: generic code embedding via lexical and program dependence analyses. Proc – 2022 Min Softw Repositories Conf MSR 2022 524–536. https://doi.org/10.1145/3524842.3528456
Marjanov T, Pashchenko I, Massacci F (2022) Machine learning for source code vulnerability detection: what works and what isn't there yet. IEEE Security & Privacy 20:60–76. https://doi.org/10.1109/MSEC.2022.3176058
Nguyen V, Le T, de Vel O et al (2020) Dual-Component deep domain adaptation: A new approach for cross project software vulnerability Detection. In: advances in knowledge discovery and data mining. Springer, Cham
Nguyen V, Le T, Tantithamthavorn C et al (2024) Deep domain adaptation with max-margin principle for cross-project imbalanced software vulnerability detection. ACM Trans Softw Eng Methodol 33:1–34. https://doi.org/10.1145/3664602
Rothenberger JC, Diochnos DI (2023) Meta Co-Training. Two Views are Better than One
Zeng P, Lin G, Pan L (2020) Software vulnerability analysis and discovery using deep learning techniques: a survey. IEEE Access 8:197158–197172. https://doi.org/10.1109/ACCESS.2020.3034766
Zhang C, Liu B, Xin Y, Yao L (2023) CPVD: cross project vulnerability detection based on graph attention network and domain adaptation. IEEE Trans Software Eng 49:4152–4168. https://doi.org/10.1109/TSE.2023.3285910
Zhou W, Zhu X, Han QL et al (2025) The security of using large language models: a survey with emphasis on ChatGPT. IEEE/CAA Journal of Automatica Sinica 12:1–26. https://doi.org/10.1109/JAS.2024.124983
Zhou Y, Liu S, Siow J et al (n.d.) Devign: effective vulnerability identification by learning comprehensive program semantics via Graph Neural Networks. https://doi.org/10.5555/3454287.3455202
Zhu X, Wen S, Camtepe S, Xiang Y (2022) Fuzzing: a survey for roadmap. ACM Comput Surv. https://doi.org/10.1145/3512345
Zhu X, Zhou W, Han QL et al (2025) When software security meets large language models: a survey. IEEE/CAA Journal of Automatica Sinica 12:317–334. https://doi.org/10.1109/JAS.2024.124971

Radowanul Haque received his MSc degree in Computer Science in 2023 and is currently pursuing a PhD in Computer Science at Ulster University, UK. His research interests include intelligent software engineering, cybersecurity, program analysis, and natural language processing.

Aftab Ali is a Lecturer in the School of Computing at Ulster University in Belfast, United Kingdom. With over a decade of experience in academia, he has held lecturer and researcher positions at various universities. His research interests span across several areas, including cybersecurity, trust management in the Internet of Things (IoT), blockchains, digital twins, body area networks, Artificial intelligence, and machine learning applications. Dr. Ali has made significant contributions to his field through publications in scholarly international journals and conferences. Additionally, his work has led to the filing of seven patents that focus on enhancing software productivity and cybersecurity using AI. Currently, Dr. Ali serves as the lead researcher on two projects, namely Software Bug Prediction and Future IoT Security, at BT Ireland Innovation Centre (BTIIC). He leads a team of graduate students, supervising both MSc and PhD candidates, who work under his guidance and support. Dr. Ali's expertise is also sought after in the publishing domain, where he serves as a reviewer and guest editor for several international journals. Overall, Dr. Aftab Ali's work and contributions in the field of computing, cybersecurity, and AI have established him as a respected researcher, lecturer, and mentor in the academic community.

Sally McClean received her first degree in Mathematics from Oxford University, and then obtained a MSc in Mathematical Statistics and Operational Research from Cardiff University, followed by a PhD on Markov and semi-Markov models at Ulster University. She is currently Professor of Mathematics at Ulster University. Her main research interests are in Stochastic Modelling and Optimisation, particularly for Healthcare Planning, and Computer Science, specifically Process Mining, Databases, Internet of Things, Sensor Technology and Telecommunications. She has been grant holder on over £13 million worth of funding, mainly from the EPSRC, Industry, the EU and charities. Sally is a Fellow of the Royal Statistical Society, Fellow of the Operational Research Society, Fellow of the Institute of Mathematics and its Applications, past President of the Irish Statistical Association and Member of the IEEE. She has published over six hundred research papers and was previously a recipient of Ulster University's Senior Distinguished Research Fellowship.

Naveed Khan received the M.S. degree in Computer Science from King Saud University, Saudi Arabia, in 2012, and the Ph.D. degree in Computer Science from Ulster University, UK in 2018. He is currently a Lecturer with the School of Computing at Ulster University. He has extensive experience in applying machine learning, data analytics, and artificial intelligence to pervasive computing, cybersecurity, and healthcare applications. His scholarly work has been published in reputable international journals and conferences, establishing his reputation in the field. His research interests include change detection in human activities, predictive process monitoring, federated learning, anomaly detection in cyber-physical systems, Explainable AI and AI-driven healthcare solutions.